LELEC2531 – FIRST PROJECT REPORT

Submit this report no later than October 25th, 2026 at 23:59.

Group number	Name 1	Name 2	Name 3
	Maxime Michelet	Hippolyte Chevalier	
NOMAs:	58382300		

Q1 - Context (1 pt): Explain in which way building an accelerator for your project topic is useful. You can, for instance, link it to potential applications in different domains.

We're building a multiply-accumulate (MAC), it is the sum of the previous output and the product of the two inputs. MAC operations, $S_{n+1} = S_n + A_n B_n$, are repeated in neural-network matrix multiplication, convolution layer and dot products. A dedicated MAC accelerator performs the multiply and add operation directly in the hardware, potentially reducing the processor cost and speed-up the tasks.

Q2 - Chosen Topology (1 pt): Detail the accelerator topology that you have chosen for this first version. For example, draw a schematic or a block-level diagram of the circuit you have implemented, and detail the main characteristics of each block.

We propose a signed 16-bit * 16-bit Mac with a 32-bit result. The input is a 32-bit and packs A=input[31:16] and B=input[15:0]. The product is optionally arithmetically right shifted by s ϵ [0,32] before incrementing the previous output

Input: 32-bit; writing wr_input captures two operands and start one operation

CFG0: 8-bit; 0 select accumulate / replace, 1 request to clear the memory output

CFG1: 8-bit; bits [4:0] select the shift

Output: 32-bit; store the accumulated result

Timing: Capture on a rising clock edge, update output on the next rising edge (one clock period between these edges). Arithmetic overflow wraps modulo 2^32; no saturation.

Q3 – First synthesis and simulations (2 pt): Report here the first simulation results of your circuit. For instance, you can explain which test bench you used and include a screenshot of the main simulation results to assess whether the circuit works properly.

A first SystemVerilog module (mac_core.sv) and a self-checking testbench (tb_mac_core.sv) have been prepared. The testbench applies registered inputs and compares results against expected arithmetic values. Examples of expected outputs (not observed simulation results) are:

Stimulus	Expected result
3 × 4, accumulator initially 0	S = 12
Then (−2) × 5 in accumulate mode	S = 12 − 10 = 2
ASR1(−3 × 3) after clear	S = −5
Four successive additions of 2^30	S = 0 after 32-bit overflow (wrap)

The relatively high pin count reflects the standalone top-level exposing the input, output, and configuration signals; it does not represent a fully integrated board interface. A successful synthesis confirms hardware mapping, not correct numerical operation. RTL simulation: pending. We prepared a self-checking tb_mac_core.sv testbench for ModelSim/Questa. It will check, among other cases, 3 × 4 → 12, accumulation of (−2) × 5 → 2, signed shifts, clearing, replace mode, and overflow wrapping. These are expected values; no passing simulation is claimed yet. A ModelSim waveform can be added below after execution




```
# PASS 3 * 4                    result=12 (0x0000000c)
# PASS 12 + (-2)*5              result=2 (0x00000002)
# PASS shift positive           result=6 (0x00000006)
# PASS shift negative           result=1 (0x00000001)
# PASS replace mode             result=14 (0x0000000e)
# PASS signed edge case         result=1073741824 (0x40000000)
# PASS overflow step 1          result=1073741824 (0x40000000)
# PASS overflow step 2          result=-2147483648 (0x80000000)
# PASS overflow step 3          result=-1073741824 (0xc0000000)
# PASS overflow wraps to zero   result=0 (0x00000000)
# ALL TESTS PASSED
# ** Note: $finish    : C:/Users/maxou/OneDrive/Documents/Etudes/Q7/LELEC2531 - Digital electronic systems/PROJET1/LELEC2531_MAC_starter/tb_mac_core.sv(97)
```

Q4 - Perspectives to improve your accelerator (1 pt): Make a quick list of possible improvements that you would do to increase the performance of your accelerator. These improvements can be on the architecture, the design itself or some blocks in particular

• Parallel INT8 MACs: compute two 8-bit products per 32-bit input word to increase work done per accepted input, with precision trade-offs.

• Extra pipeline stage: register the product before accumulation to shorten the combinational critical path and potentially improve clock frequency (at the cost of added latency).

• Configurable arithmetic: compare fixed-point scaling and saturation/rounding against the baseline's truncation and wrap-around behavior.

• Quantitative comparison: measure latency, throughput, logic elements, embedded multipliers, timing and estimated power after synthesis; compare each optimization with the baseline.

Report about the use of AI tools (list your different usage of the tools, whether in designing the accelerator or creating the report).

AI tool used: ChatGPT (OpenAI).
Uses in the accelerator design :

- Discussion and proposal of a baseline architecture
- Assistance in drafting an initial SystemVerilog implementation
- Guidance on Quartus
- Helped to write a report